

Sistemas de recomendación

Arias, Blanc, Comelli.

2026

1. Introducción

No es novedad que hoy los sistemas de recomendación son parte de nuestra vida cotidiana. En las redes sociales, en la publicidad segmentada e incluso en las plataformas de Streaming. Esto no es para nada grave, quien no ha visto una película recomendada por una plataforma que resultó ser muy buena? En general los sistemas de recomendación suelen ser muy útiles tanto para los usuarios que reciben el contenido que les interesa como para los proveedores que pueden dirigir sus productos a un público en particular. Los sistemas de recomendación son muy utilizados y francamente exitosos en el mundo comercial pero, ¿podremos ampliar su uso con un enfoque macroeconómico? Supongamos por un momento que somos un país productor y tenemos países que son habituales consumidores de nuestras exportaciones. Naturalmente, uno buscaría mantener la competitividad colocando nuevos productos o ampliando la cartera de países importadores, pero esto puede ser muy costoso y además, en algunos casos, no está tan claro que países son potenciales clientes. Es en este punto donde los **sistemas de recomendación** capturando la información necesaria, pueden ayudar a predecir países interesados en nuestros productos. En particular, se discutirán y se expondrán modelos de **factores latentes**

2. Preliminares/Motivación

Dada una matriz R , de $n \times m$, no siempre vamos a querer (o poder) factorizarla por ser muy costosa. Es natural, entonces, buscar una aproximación.

Es decir, busquemos a priori, U y V tales que,

$$R \approx UV.$$

Para dar cuenta de qué tan buena es nuestra aproximación, hacemos uso de la norma de Frobenius (o equivalentes). Por lo tanto, queremos U, V tales que la diferencia sea tan pequeña como sea posible, es decir,

$$\|R - UV\|_F.$$

Podemos pensarlo, a su vez, como un problema de optimización, donde queremos minimizar $J = \|R - UV\|_F$.

Nuevamente, dependiendo de cómo es la naturaleza de las entradas de la matriz R , pediremos que U, V tengan ciertas propiedades. Por ejemplo, si queremos que nuestras matrices sean ortonormales entre sí y que sean del menor rango posible, podemos hacer uso de la *SVD* (singular value decomposition).

Ahora bien, nuestra matriz R no se trata de números sin significado alguno, en R hay codificados datos sobre la realidad, por ejemplo, podríamos pensar que las filas de R son usuarios de netflix y las columnas son películas. Aquí los datos r_{ij} serían o bien si el usuario ha visto una película o bien cuanta puntuación ha dado un usuario a una película. Tomando este ultimo caso, r_{ij} será la puntuación del usuario i en la película j .

Nuestro objetivo no se trata de aproximar R , pues si ya conozco R no hay nada que aproximar. El problema es justamente ese. Un usuario puede ver 10, 20 o quizás 100 películas pero hay miles, por lo tanto en la matriz R habrán pocos datos conocidos (las puntuaciones de las películas que vio) y muchísimos datos desconocidos (puntuaciones de películas que no vio). La idea de los sistemas de recomendación es poder ser capaz de determinar, a través de los datos conocidos, es inferir el valor de los datos desconocidos, en nuestro ejemplo, cuales películas que el usuario aún no ha visto, le podrían gustar.

Para dar respuesta a esta pregunta hay que, en primer lugar, rellenar las entradas vacías, pues no podemos trabajar con matrices que le faltan datos. Hemos de rellenar las matrices de manera coherente, dependiendo de la naturaleza del objeto.

Para comenzar con este viaje de recomendaciones, veremos un método sencillo y algo costoso pero que funciona bien.

3. Conceptos latente

Asumiendo que los datos poseen ciertas características, a priori no evidente, que pueden compartir entre sí, será de gran utilidad poder extraerlas. Más aún nos permitirán estudiar las preferencias de los usuarios a ciertas características y la tendencia de los items a cumplirlas. A estas características ocultas es a lo que denominaremos como *conceptos latentes*.

Dado que la entrada i, j de nuestra matriz de datos R representa la valoración del usuario i sobre el item j y sabiendo que en una factorización donde $R = UV$ el elemento r_{ij} es tal que

$$r_{ij} = \vec{u}_i \cdot \vec{v}_j$$

donde $\vec{u}_i$ es la fila i de la matriz U y $\vec{v}_j$ es la columna j de V , entonces la valoración del usuario i sobre el item j depende tanto de U como de V . Recordando que

Supongamos que tenemos una matriz conocida R de tamaño $m \times n$ que representa las valoraciones de m usuarios sobre n items. En tal caso podemos utilizar diferentes métodos para factorizarla tal que

$$R = UV^T$$

con U de tamaño $m \times k$ y V de $n \times k$ donde k es el rango de la matriz R .

Notemos que $r_{ij} = \vec{u}_i \cdot \vec{v}_j$ donde $\vec{u}_i$ representa la fila i de U y $\vec{v}_j$ la fila j de la matriz V . Para nuestro propósito, podemos pensar que cada coordenada del vector $\vec{u}_i$ (de dimensión k) representa la afinidad del usuario i al concepto latente correspondiente.

De la misma forma podemos pensar que el vector $\vec{v}_j$ también de dimensión k representa la afinidad del item j al concepto latente correspondiente. Así, llamamos **factores latentes** a los vectores $\vec{u}_i$ y $\vec{v}_j$. Incluso si el rango de R fuese mayor que k , podríamos aproximarla tal que

$$R \approx UV^T$$

con U y V de rango k . De esta manera, podemos calcular su error utilizando la norma de Frobenius tal que

$$\|R - UV\|_F$$

Podemos pensar la aproximación, a su vez, como un problema de optimización, donde queremos minimizar $J = \|R - UV\|_F$.

Sin embargo, el objetivo de nuestro estudio es predecir datos faltantes de nuestra matriz de valoraciones. En este contexto no tiene sentido computar la norma de Frobenius habida cuenta de que nos faltan entradas. Pero, aun así podemos reescribir la función objetivo en término de los datos observados. En virtud de esto, definamos entonces

$$S = \{(i, j) : r_{ij} \text{ es conocido}\}.$$

Supongamos que tenemos una aproximación lo suficientemente buena de R mediante el producto UV^T , entonces, podríamos predecir las entradas faltantes de la forma

$$\hat{r}_{i,j} = \vec{u}_i \cdot \vec{v}_j.$$

donde $i, j \notin S$.

Notemos que la diferencia entre las entradas conocidas (i, j) y la predicción está dada por

$$e_{ij} = (r_{ij} - \hat{r}_{ij}) = r_{ij} - (\vec{u}_i \cdot \vec{v}_j).$$

Se sigue, entonces, que el problema de minimización a resolver es

$$\text{Min} \frac{1}{2} \sum_{i,j \in S} e_{ij}^2.$$

4. Gradiente Descendiente

5. Gradiente Descendiente Estocástico

6. Regularización

Uno de los principales problemas a la hora de trabajar con matrices dispersas

Puede ocurrir, con matrices dispersas, que nuestra aproximación sea teóricamente buena, en el sentido de que el error de nuestro problema (de minimización) sea muy pequeño pero a la luz de los hechos, las predicciones de los no observados no son acordes a lo que uno esperaría. Por ejemplo, si estamos en el contexto de una matriz de calificaciones de usuarios sobre películas, sería anormal que se recomiende una película de Terror a quien tiene preferencias claras por películas de Comedia.

Para aliviar estos efectos, haremos uso de la *regularización Tikhonov*. Esencialmente buscamos controlar mediante un factor $\lambda > 0$ aquellas predicciones sobre valores observados que sean grandes, favoreciendo soluciones más simples.

(Agregar una comparación grafica entre la distribucion de las predicciones con y sin regularizacion)

Matemáticamente, esto es bla bla bla.

$$\text{Min } J = \frac{1}{2} \sum_{(i,j) \in S} e_{ij}^2 + \frac{\lambda}{2} \sum_{i,s} u_{is}^2 + \frac{\lambda}{2} \sum_{j,s} v_{js}^2.$$

7. Gradiente Descendiente con Regularización

8. Gradiente Descendiente Estocástico con Regularización

9. Gradiente Descendente y Descendente estocástico.

Tabla 1: Convergence and Performance Comparison ($K = 2$ vs. $K = 8$)

Algorithm	K	Iterations	Final RMSE	RMSE Change (Δ)	Status
Batch GD	2	647	1,2855	$3,7 \times 10^{-5}$	Converged
SGD (Aggarwal 2016)	2	65	0,7445	$1,0 \times 10^{-5}$	Converged
Batch GD (w/ reg.)	2	1000	1,2442	—	Max Iterations
SGD (w/ reg.)	2	39	0,8009	$1,3 \times 10^{-5}$	Converged
Batch GD	8	1000	0,6614	—	Max Iterations
SGD (Aggarwal 2016)	8	56	0,5905	$1,7 \times 10^{-5}$	Converged
Batch GD (w/ reg.)	8	1000	0,6691	—	Max Iterations
SGD (w/ reg.)	8	100	0,6509	$5,5 \times 10^{-5}$	Converged

Tabla 2: Esquema del experimento para 10-fold cross-validation.

Parametros	Valor
Tamaño de la matriz	610×9724
Valores observados	100 836 (1.70 % dense)
Número de folds	10
Factores elegidos k	2, 5, 8, 10, 20, 30, 50
λ	0.01
γ	0.01
Iteraciones maximas	1000
Tolerancia	10^{-4}

Tabla 3: Resultados de la validación cruzada, listados por RMSE. The generalization gap is the mean test RMSE minus the mean train RMSE.

k	λ	Test RMSE	SD	Train RMSE	SD	Iteraciones	Convergencia	Tiempo
2	0.01	1.0893	0.0100	0.7463	0.3431	47	10/10	6m 33.1s
5	0.01	1.1060	0.0129	0.6393	0.4667	60	10/10	8m 24.1s
8	0.01	1.1367	0.0125	0.5680	0.5688	67	10/10	9m 50.7s
10	0.01	1.1451	0.0123	0.5278	0.6173	68	10/10	9m 52.3s
50	0.01	1.1891	0.0117	0.1545	1.0346	108	10/10	16m 51.4s
20	0.01	1.1907	0.0182	0.3726	0.8181	85	10/10	12m 21.3s
30	0.01	1.1999	0.0096	0.2748	0.9250	90	10/10	13m 39.5s

Tabla 4: Test RMSE per fold for each number of factors k ($\lambda = 0,01$).

Fold	Number of factors k						
	2	5	8	10	20	30	50
1	1.1014	1.1204	1.1484	1.1624	1.2143	1.2176	1.2005
2	1.0882	1.1042	1.1412	1.1379	1.1826	1.2004	1.1906
3	1.1006	1.1238	1.1562	1.1561	1.2061	1.2070	1.2019
4	1.0831	1.1003	1.1283	1.1314	1.1743	1.1944	1.1856
5	1.1018	1.1163	1.1509	1.1655	1.2169	1.2050	1.2103
6	1.0729	1.0895	1.1178	1.1323	1.1624	1.1890	1.1728
7	1.0861	1.0957	1.1303	1.1350	1.1798	1.1994	1.1869
8	1.0794	1.0869	1.1311	1.1420	1.1775	1.1981	1.1820
9	1.0849	1.1081	1.1244	1.1447	1.1989	1.1836	1.1783
10	1.0948	1.1148	1.1387	1.1435	1.1939	1.2037	1.1825
Mean	1.0893	1.1060	1.1367	1.1451	1.1907	1.1999	1.1891
SD	0.0100	0.0129	0.0125	0.0123	0.0182	0.0096	0.0117

Tabla 5: Final train RMSE per fold for each number of factors k ($\lambda = 0,01$). The initial train RMSE was approximately 3.653 in every fold.

Fold	Number of factors k						
	2	5	8	10	20	30	50
1	0.7418	0.6375	0.5636	0.5297	0.3588	0.2688	0.1475
2	0.7538	0.6376	0.5646	0.5280	0.3752	0.2739	0.1485
3	0.7485	0.6336	0.5645	0.5216	0.3694	0.2746	0.1531
4	0.7399	0.6412	0.5660	0.5325	0.3829	0.2801	0.1484
5	0.7416	0.6362	0.5621	0.5228	0.3607	0.2771	0.1529
6	0.7532	0.6369	0.5747	0.5325	0.3808	0.2686	0.1603
7	0.7525	0.6464	0.5717	0.5269	0.3876	0.2818	0.1571
8	0.7409	0.6595	0.5680	0.5211	0.3741	0.2702	0.1570
9	0.7560	0.6316	0.5742	0.5284	0.3643	0.2928	0.1464
10	0.7344	0.6325	0.5699	0.5340	0.3720	0.2601	0.1742
Mean	0.7463	0.6393	0.5680	0.5278	0.3726	0.2748	0.1545

Tabla 6: Epochs until convergence per fold for each number of factors k ($\lambda = 0,01$). All 70 fits converged.

Fold	Number of factors k						
	2	5	8	10	20	30	50
1	55	60	75	65	111	99	124
2	36	60	72	71	81	89	119
3	41	70	69	75	85	90	107
4	49	53	68	60	70	81	121
5	55	61	74	71	103	85	109
6	37	64	60	63	75	100	97
7	36	48	62	67	65	81	101
8	55	38	69	80	81	94	102
9	35	73	59	69	98	70	125
10	67	72	66	64	84	113	80
Mean	47	60	67	68	85	90	108
Min	35	38	59	60	65	70	80
Max	67	73	75	80	111	113	125

Tabla 7: Running time per fold in seconds for each number of factors k ($\lambda = 0,01$). The last row gives the total time per combination.

Fold	Number of factors k						
	2	5	8	10	20	30	50
1	45.6	50.2	61.9	54.4	99.2	92.2	123.0
2	29.3	50.2	63.6	63.6	72.1	81.1	111.6
3	38.0	58.5	65.4	65.2	71.2	81.5	100.5
4	41.4	44.2	62.3	50.4	58.1	73.7	113.6
5	46.5	50.4	63.2	59.7	87.0	76.7	100.1
6	30.9	53.4	54.6	53.8	62.2	90.6	87.3
7	30.0	40.6	53.5	61.8	55.1	73.1	88.0
8	46.5	31.9	59.2	69.4	71.9	84.6	97.3
9	29.0	63.6	50.8	59.0	85.9	63.7	115.4
10	55.8	61.0	55.9	54.8	78.6	102.1	74.4
Total	6m 33.1s	8m 24.1s	9m 50.7s	9m 52.3s	12m 21.3s	13m 39.5s	16m 51.4s

Tabla 8: SGD refit on all 100 836 observed ratings with the best combination ($k = 2$, $\lambda = 0,01$, $\gamma = 0,01$). Total drop in RMSE: 2.9080 (79.6 %); run time 52.7 s.

Iteration	Train RMSE	Change in RMSE
0 (init)	3.6535	—
10	0.8364	0.014911
20	0.7779	0.003289
30	0.7613	0.000408
40	0.7530	0.000555
50	0.7480	0.000394
55 (converged)	0.7455	0.000073

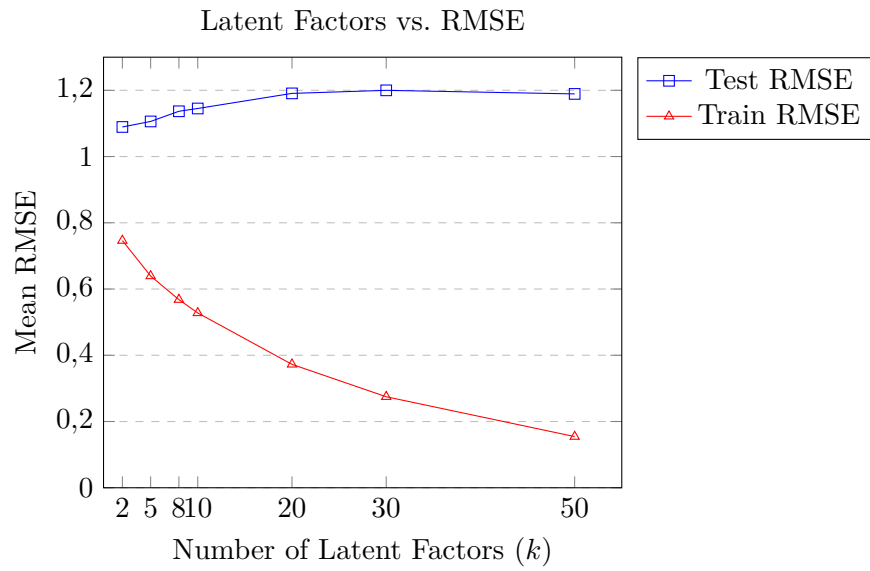


Figura 1: Model error and generalization gap as a function of latent factors based on results2.txt.

Referencias

- [1] C. Aggarwal, C. Procopiuc y P. S. Yu. «Finding localized associations in marketbasket data». En: *IEEE Transactions on Knowledge and Data Engineering* 14.1 (2001), págs. 51-62.
- [2] C. C. Aggarwal. *Recommender Systems: The Textbook*. Springer International Publishing, 2016. DOI: [10.1007/978-3-319-29659-3](https://doi.org/10.1007/978-3-319-29659-3).
- [3] M. P. Deisenroth, A. A. Faisal y C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [4] T. Hastie, R. Tibshirani y J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer Series in Statistics. Springer, 2009.
- [5] G. James, D. Witten, T. Hastie y R. Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. 2nd. Springer, 2021.
- [6] J. Nocedal y S. J. Wright. *Numerical Optimization*. Springer, 1999.